Every Boolean function admits a read-four permutation branching program of length four n minus four

James Cook, Ian Mertz

doi:10.4230/LIPIcs.CCC.2022.8; CC BY 4.0

Editorial note. Complete original published new upper-bound proof retained with the full foundational polynomial/register algorithm and model conversion. Older upper bounds are context only and not additional counted results. Original mathematics, assumptions and formulas unchanged; long displayed register update split over lines for page width. EarlierCC040 files not modified; new independent fragments added onYupeng only.

Allowed standard prerequisites (editorial).

- Elementary permutation identities and Boolean multilinear polynomial interpolation. Complete source-local polynomial representation, transparent algorithm, model conversion and permutation lemmas included; cited older read4 constructions are comparison context, not substituted proofs

2 Preliminaries

We introduce two space-bounded models of computation. Our first model is a variation of branching programs, which are the standard syntactic (i.e. non-uniform) notion of space-bounded computation (see [7]).

Definition5 (m-catalytic branching program [8]). Let $n \in \mathbb{N}$ and let $f : \{0,1\}^n \to \{0,1\}$ be an arbitrary function. An m-catalytic branching program is a directed acyclic graph G with the following properties:

- There are m source nodes and $2m$ sink nodes.
- Every non-sink node is labeled with an input variable x_i for $i \in [n]$, and has two outgoing edges labeled0 and1.
- For every source node v there is one sink node labeled with $(v, 0)$ and one with $(v, 1)$.

We say that G computes f if for every $x \in \{0,1\}^n$ and source node v, the path defined by starting at v and following the edges labeled by the value of the x_i labeling each node ends at the sink labeled by $(v, f(x))$. The size of G is

the number of non-sink nodes[1] in G. We also consider m-catalytic branching programs with standard restrictions:

- layered branching programs: for some $\ell \in \mathbb{N}$, there exists a function $\sigma : G \to [\ell+1]$ such that for all $u \in G$, the outgoing edges of u go to nodes v, w such that $\sigma(v) = \sigma(w) = \sigma(u) + 1$; we call the set of nodes $\{v \in \sigma^{-1}(j)\}$ the jth layer. Furthermore for each $j \in [\ell]$ there exists an input variable x_{j_i} which is the variable labeling every $v \in \sigma^{-1}(j)$.[2] The width of G is $\max_{j\in[\ell]} |\sigma^{-1}(j)|$ and the length of G is ℓ. Note that the size of G is at most the product of the length and width of G.

- read-k branching programs: for any start node v and any input x, each variable x_i is read at most k times during the computation starting at v. We note that for layered branching programs, this corresponds to each variable x_j labeling at most k layers.

- permutation branching programs: consider a layered branching program of width $2m$ with $2m$ source nodes $S = \{s_1 \dots s_{2m}\}$ and $2m$ sink nodes $T = \{t_1 \dots t_{2m}\}$, and let $P(x) : [2m] \to [2m]$ be the function such that $P(x)(i) = i'$ where the computation path starting from s_i on input x goes to $t_{i'}$. Then there exists a permutation $\sigma^* : [2m] \to [2m]$ such that $P(x)$ is the identity function when $f(x) = 0$ and $P(x) = \sigma^*$ when $f(x) = 1$. This is not an m-catalytic branching program in the strict sense, and we will address this model more precisely in Section4.

Our second model comes from a line of work starting with [3], more recently fleshed out in [4] and used in many follow-up works on catalytic computation [5,6].

Definition6 (Transparent register program). Let R be a ring and $f \in \{0,1\}^n \to \{0,1\}$ a function. A register program P is defined by a set of registers $\mathcal{S} = \{R_1 \dots R_s, R_{\text{out}}\}$, each storing a value in R, plus an ordered list of t instructions where for every $j \in [t]$ the jth instruction is $R \leftarrow R + p_j(x_i, \mathcal{S} \setminus \{R\})$ for some $i \in [n]$, $R \in \mathcal{S}$, and polynomial $p_j \in R[x, y_0 \dots y_s]$. We say that P computes f if for every $x \in \{0,1\}^n$, after initializing $R = 0$ for all registers R and then executing all instructions in order, the value stored in R_{out} is $f(x)$. We say that P transparently computes f if instead of initializing all R to0, each R begins in an arbitrary initial state τ_i, and at the end of the program we have $R_{\text{out}} = \tau^{\text{out}} + f(x)$ and $R_i = \tau_i$ for all i. The size of P is the number of registers $s + 1$ and the time of P is the number of instructions t. We use register programs as an abstraction for understanding m-catalytic branching programs, which will be the model our main results will refer to. The two models are connected through the following observation (see Figure2 of the original article for an example of our construction).

[1] This is somewhat non-standard, but when talking about layered branching programs this simplifies things by defining the length as the number of times we read variables, which will in turn be connected to the number of instructions in our register program model. This choice does not affect the asymptotics of any results.

[2] By construction layer $\ell + 1$ will contain exactly the sink nodes of G. See the previous footnote for an explanation of this somewhat non-standard convention.

Observation7. Let $f_n : \{0,1\}^n \to \{0,1\}$ be a family of functions and let P_n be a family of register programs over a family of rings R_n with size $s(n)+1$ and time $t(n)$ each transparently computing f_n. Then f_n can be computed by a family of m-catalytic branching programs of width $|R_n|m$ and length $t(n)$, where $m = |R_n|^{s(n)}$.

Proof. We will define a branching program with width $|R_n|^{s(n)+1}$ and length $t(n)$. Each layer will represent a stage in the register program and each node in a layer will represent a setting to the registers at that time. Since each register program step only requires us to read one input variable, at layer k we query the variable associated with step k in the register program and create edges from each node in layer k to the nodes at layer $k+1$ representing the state of our memory after the step has completed. We label each input node as τ for some distinct initial configuration $\tau = (\tau_1 \dots \tau_{s(n)})$ to all registers except R_{out}, and we treat R_{out} as being initialized to0. Then by the fact that P transparently computes f, starting at node τ we are guaranteed to reach a node $(\tau, f(x))$. Since in each layer we have a node for every setting of the $s(n)+1$ registers, the width of our branching program is $|R_n|^{s(n)+1}$, and since each non-output layer corresponds to a unique instruction from our register program, the length of our branching program is $t(n)$. □

Note8. In our computation we often include instructions of the form $R \leftarrow R + p_j(R/\{R\})$, i.e. instructions that do not require reading a variable x_i. By observing the above proof, it is clear that such instructions do not add any length to the branching program, as they can be computed at the same time as an adjacent instruction.

3 Saving Space

In these sections, our register programs will all operate over the field of two elements: $R_n = \mathbb{F}_2$ for all n.

3.1 Basic Algorithm

In this section, we will prove:

Theorem9. For any function $f : \{0,1\}^n \to \{0,1\}$ there is an m-catalytic branching program with length $4n$ and width $2m$ that computes f, where $m = 2^{n+2^n}$. This is proved by Algorithm1 below. This nearly reproduces Potechin's Theorem3.1[11], but with a worse value $m = 2^{n+2^n}$ instead of 2^{2^n-1}. Nonetheless, we will find it a useful starting point for our algorithm that trades space for time in Section3.2.

Proof. We will design a program that uses $n+2^n$ work registers plus one output register R_{out}, which is sufficient by Observation7. First, we have n registers $R_1^{\text{in}}, \dots, R_n^{\text{in}}$, corresponding to the n input bits. This correspondence is given by the following subroutine:

ToggleInput.

1: **procedure** ToggleInput

2: **for** $i = 1, \ldots, n$ **do**

3: $R_i^{\text{in}} \leftarrow R_i^{\text{in}} + x_i$

4: **end for**

5: **end procedure**

After ToggleInput runs, the registers have values $R_i^{\text{in}} = \tau_i^{\text{in}} + x_i$, where τ_i^{in} stands for the initial value of R_i^{in}. If we run it a second time, the registers are restored to their original values: $R_i^{\text{in}} = \tau_i^{\text{in}}$. Since we query all n variables once, ToggleInput requires time n to run once. Before defining our other 2^n registers, we introduce an algebraic view of f, which will be our main focus. We can view f as a multilinear polynomial $p_f \in \mathbb{F}_2[x_1, \ldots, x_n]$ using basic interpolation:

Lemma10. For any function $f : \{0,1\}^n \to \{0,1\}$ there is a multilinear polynomial $p_f \in \mathbb{F}_2[x_1, \ldots, x_n]$ such that $p_f(\vec{x}) = f(\vec{x})$ for all $\vec{x} \in \{0,1\}^n$.

Proof. For any $\vec{y} \in \{0,1\}^n$, we can algebraically define the indicator function $[\vec{x} = \vec{y}]$ as $\prod_{i=1}^n (x_i + y_i + 1) \in \mathbb{F}_2[x_1, \ldots, x_n]$. Then we can set

$$p_f = \sum_{\vec{y} \in \{0,1\}^n : f(\vec{y}) = 1} [\vec{x} = \vec{y}].$$

□

Now define $y_i = \tau_i^{\text{in}} + x_i$; in other words, y_i is the value of R_i^{in} after running ToggleInput. Define $q_f(y_1, \ldots, y_n, \tau_1^{\text{in}}, \ldots, \tau_n^{\text{in}}) = p_f(y_1 - \tau_1^{\text{in}}, \ldots, y_n - \tau_n^{\text{in}})$ to be the result of rewriting p_f using the y_i and τ_i variables.[3] For $S, S' \subseteq [n]$, let $c_{S,S'}$ be the coefficient of $(\prod_{i \in S} \tau_i^{\text{in}}) \cdot (\prod_{i \in S'} y_i)$ in q_f, so that

$$q_f(\vec{\tau}^{\text{in}}, \vec{y}) = \sum_{S, S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{i \in S'} y_i \right).$$

Note that S and S' are disjoint whenever $c_{S,S'} \neq 0$, because p_f is multilinear. We now introduce our other registers: we have 2^n registers R_S indexed by subsets $S \subseteq [n]$. Our next subroutine prepares us to use these registers to compute q_f:

ToggleMonomials.

1: **procedure** ToggleMonomials

2: ToggleInput

3: **for** $S' \subseteq [n]$ **do**

4: $R_{S'} \leftarrow R_{S'} + \prod_{i \in S'} R_i^{\text{in}}$

[3]For example, if f is the OR function $f(x_1, x_2) = x_1 \vee x_2$, we have $p_f(x_1, x_2) = x_1 + x_2 + x_1 x_2$ and $q_f(\tau_1^{\text{in}}, \tau_2^{\text{in}}, y_1, y_2) = y_1 + \tau_1^{\text{in}} + y_2 + \tau_2^{\text{in}} + y_1 y_2 + y_1 \tau_2^{\text{in}} + \tau_1^{\text{in}} y_2 + \tau_1^{\text{in}} \tau_2^{\text{in}}$, and both are equal to $f(x_1, x_2)$ so long as $\vec{y}$ have the correct values.

5: **end for**

6: ToggleInput

7: **end procedure**

After ToggleMonomials runs, we have $R_S = \tau_S + \prod_{i \in S} y_i$ for each $S \subseteq [n]$, where τ_S stands for the register's initial value. The R^{in} registers have their initial values $R_i^{\text{in}} = \tau_i^{\text{in}}$. We run ToggleInput twice and have 2^n additional instructions, but since the additional instructions do not query any x variables they can be computed in the last x query of ToggleInput, for a total runtime of $2n$. Our final algorithm for computing f is:

Algorithm1: Transparently compute f in time $4n$ with $n + 2^n + 1$ registers.

1: ToggleMonomials

2: $R_{\text{out}} \leftarrow R_{\text{out}} + \sum_{S,S' \subseteq [n]} c_{S,S'} (\prod_{i \in S} R_i^{\text{in}}) R_{S'}$

3: ToggleMonomials

4: $R_{\text{out}} \leftarrow R_{\text{out}} - \sum_{S,S' \subseteq [n]} c_{S,S'} (\prod_{i \in S} R_i^{\text{in}}) R_{S'}$

When Line2 executes, we have $R_{S'} = \tau_{S'} + \prod_{i \in S'} y_i$, so after that line,

$$R_{\text{out}} = \tau^{\text{out}} + \sum_{S,S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\tau_{S'} + \prod_{i \in S'} y_i \right).$$

Then when Line4 executes, we have $R_{S'} = \tau_{S'}$, so the final value is

$$R_{\text{out}} = \tau^{\text{out}} + \sum_{S,S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{i \in S'} y_i \right)$$
$$= \tau^{\text{out}} + f(x_1, \ldots, x_n).$$

So Algorithm1 correctly computes f. The space of the program is $n + 2^n$ by construction, and as before we can ignore the instructions on lines2 and4 since they do not use $\mathbf{x}$, giving us a total runtime of $4n$ from the two calls to ToggleMonomials. □

4 Saving Time

In the previous section we took the length $4n$ branching programs of [11,12] as a starting point to analyze whether m could be significantly reduced while still maintaining a linear amortized size. In this section we investigate the opposite question: namely, is $4n$ optimal? If we do not restrict the amortized size of our program, then every function has a branching program of length n regardless of m. We will not only consider branching programs of linear amortized size, but the stricter model of permutation branching programs. We focus on permutation branching programs in this section for two reasons. First, all our algorithms in the previous section can be converted to permutation

branching programs. To see this, we resist our connection between register programs and m-catalytic branching programs, as proven in Observation7. Our observation in fact says something stronger, which is that f_n can be computed by a family of permutation branching programs of width $2^{s(n)+1}$. This follows because we can choose to not fix R_{out} to be0, and instead have one source node corresponding to each initial configuration $\tau_1 \dots \tau_s, \tau^{\text{out}}$; by Definition6 this source reaches the sink node labeled $\tau_1 \dots \tau_s, \tau^{\text{out}} + f(x)$, which is the identity permutation when $f(x) = 0$ and a fixed set of $2^{s(n)}$ transpositions otherwise. Thus as a corollary of [11,12] we get the following:

Theorem16. Every function f can be computed by a read-4 permutation branching program of width 2^{2^n-1} (or $2^{\binom{n}{\leq d}-1}$, where d is the $\mathbb{F}_2$-degree of p_f). Given the strength of these results, only a factor of4 away from the best conceivable amortized size, there is no reason a priori to believe that permutation branching programs are too weak to consider even better upper bounds. Indeed we will show that improvements are possible.

4.1 Notation and tools

Before going into our results, we formally define permutation branching programs along with the notation we will use in the rest of this section. These programs will have a more specialized form than when we introduced them in Section2, which we subsequently justify. We assume basic familiarity with permutations, and we write $\sigma_1\sigma_2$ as a shorthand for $\sigma_2 \circ \sigma_1$.

Definition17. Let $n, m, s \in \mathbb{N}$ and let $f : \{0,1\}^n \to \{0,1\}$ be a function such that $f(0 \dots 0) = 0$. A permutation branching program is a sequence $P = \pi_1 \dots \pi_\ell$, where each π_j is a pair $\langle i_j, \sigma_j \rangle$ where $i_j \in [0..n]$ and σ_j is a permutation of $[m]$. We refer to each π_j as an instruction of P. The width of P is m and the length of P is ℓ. For any $\alpha \in \{0,1\}^n$ we define $P(\alpha)$ as follows: fix $\sigma = \text{id}$, and for every $j = 1 \dots s$, we set σ to $\sigma\sigma_j$ if $\pi_j = \langle 0, \sigma_j \rangle$ or $\pi_j = \langle i_j, \sigma_j \rangle$ where $\alpha_{i_j} = 1$, and leave σ unchanged otherwise (that is, if $\pi_j = \langle i_j, \sigma_j \rangle$ where $\alpha_{i_j} = 0$). Our output is the final value of σ. We say that P computes f if there exists a permutation $\sigma^* \neq \text{id}$ such that $P(\alpha) = \text{id}$ if $f(\alpha) = 0$ and $P(\alpha) = \sigma^*$ if $f(\alpha) = 1$. We make three observations about our choices in Definition17. First, the restriction that $f(0 \dots 0) = 0$ will be a convenience; we can always compute $\neg f$ instead if this condition does not hold, or change Definition17 such that $P(\alpha) = \text{id}$ if $f(\alpha) = 1$ and vice versa.[4] Second, in a layer $\langle i, \sigma_j \rangle$ reading variable x_i, we only fix a permutation in the case that x_i is set to1. This is without loss of generality, as adding a layer of the form $\langle 0, \sigma'_j \rangle$ before an instruction can be thought of as choosing a permutation in the case that x_i is set to0 (while the permutation for $x_i = 1$ can be adjusted accordingly). Before going on to our third observation, we state and prove four simple lemmas which will allow us to conveniently restructure our programs P.

[4]We also note that if P computes $\neg f$, we can compute f by appending the instruction $\langle 0, (\sigma^*)^{-1} \rangle$ to P. We avoid taking this route because a later observation will allow us to remove these fixed layers, but only when $f(\alpha) = 0$, which would cause our logic to become circular.

Lemma18. Let P be a permutation branching program computing f and let j be such that $i_j = i_{j+1}$. Then the program P' resulting from replacing π_j, π_{j+1} with $\pi'_j = \langle i_j, \sigma_j\sigma_{j+1}\rangle$ is also a valid program for computing f.

Proof. In both P and P', the permutations σ_j and σ_{j+1} are both applied when $i_j = 1$ and neither are applied when $i_j = 0$. □

Lemma19. Let P be a permutation branching program computing f and let j be such that $\sigma_j\sigma_{j+1} = \sigma_{j+1}\sigma_j$. Then the program P' resulting from switching the order of π_j and π_{j+1} is also a valid program for computing f.

Proof. Consider any assignment α to $\mathbf{x}$. In the case that either α_{i_j} or $\alpha_{i_{j+1}}$ is set to0, these programs compute identical permutations as either σ_j or σ_{j+1} will not be applied. If both are set to1, then

$$P'(\alpha) = \Sigma_1\sigma_{j+1}\sigma_j\Sigma_2 = \Sigma_1\sigma_j\sigma_{j+1}\Sigma_2 = P(\alpha)$$

where Σ_1, Σ_2 are the permutations corresponding to the rest of the instructions on input α. □

Lemma20. Let $P = \pi_1 \dots \pi_s$ be a permutation branching program computing f, let $\pi_j = \langle i_j, \sigma_j\rangle$ for all j, and let $j^* \in [\ell]$ be such that $i_{j^*} = 0$. Then there exists a permutation branching program $P' = \pi'_1 \dots \pi'_{j^*-1}\pi'_{j^*+1} \dots \pi'_\ell\pi_{j^*}$ computing f, where $\pi'_j = \langle i_j, \sigma'_j\rangle$ for some permutation σ'_j.

Proof. For $j < j^*$ define $\sigma'_j = \sigma_j$, and for $j > j^*$ define $\sigma'_j = \sigma_{j^*}\sigma_j\sigma_{j^*}^{-1}$. Clearly because $\sigma_{j^*}^{-1}$ and σ_{j^*} cancel out for every adjacent pair of permutations σ'_j, $P'(\alpha)$ contains exactly the same permutations as $P(\alpha)$ in the same order regardless of the assignment α.[5] □

Our next observation is that the layers of the form $\langle 0, \sigma_j\rangle$ are only a convenience and are not necessary to our definition. Let P be our program for f and let $\sigma_1 \dots \sigma_k$ be the permutations corresponding to instructions π_j where $i_j = 0$. By our restriction that $f(0 \dots 0) = 0$ we get $\sigma_1 \dots \sigma_k = P(0 \dots 0) = \mathrm{id}$, and by Lemma20 we can move the instructions π_j with $i_j = 0$ to the end of the program, in order, at which point we can simply remove them all using Lemma18 as they compose to the identity for any input α. We can also generalize Lemma20 for restrictions of the function f, meaning when we fix the values of some variables and consider the function on the remaining variables. This is simply the observation that fixing variable x_i turns all instructions of the form $\langle i, \sigma_j\rangle$ into fixed layers $\langle 0, \sigma_j\rangle$.

Corollary21. Let $\rho \in \{0, 1, *\}^n$ and let f_ρ be the function f with x_i fixed to $\rho(i)$ wherever $\rho(i) \neq *$. Let $P = \pi_1 \dots \pi_\ell$ be a permutation branching program computing f, let $\pi_j = \langle i_j, \sigma_j\rangle$ for all j, and let $j^* \in [s]$ be such that $i_{j^*} = 0$ or $\rho(i_{j^*}) \neq *$. Then there exists a permutation branching program $P' = \pi'_1 \dots \pi'_{j^*-1}\pi'_{j^*+1} \dots \pi'_\ell\pi_{j^*}$ computing f_ρ, where $\pi'_j = \langle i'_j, \sigma'_j\rangle$ for some permutation σ'_j and $i'_j = i_j$ iff $\rho(i_j) = *$ and0 otherwise.

[5] This argument actually allows us to move π_{j^*} to any spot in the program we want, but we are content with just moving them to the end, for reasons which will become immediately clear.

Proof. Let program P'' be the result of replacing i_j with0 in each instruction $\pi_j \in P$ such that $\rho(i_j) \neq *$. Clearly this program computes f_ρ, and so applying Lemma20 to P'' completes the proof. □

Assuming that $f_\rho(0 \dots 0) = 0$, by our previous observation this allows us to remove all layers that read variables fixed by ρ. We also note that the other three lemmas hold for f_ρ with no changes. Finally, when f is the AND function, we can rotate our program by moving the first instruction to the end as many times as we like:

Lemma22. Let $P = \Pi_1, \Pi_2$ be a permutation branching program computing $\mathrm{AND}(x_1, \dots, x_n)$, where Π_1 and Π_2 are sequences of instructions. Then $P' = \Pi_2, \Pi_1$ also computes $\mathrm{AND}(x_1, \dots, x_n)$.

Proof. Let $\sigma^* = P'(1, \dots, 1)$ (that is, P' applied to the all-ones input). Referring to Definition17, to prove that P' computes AND, we must show(a) that $\sigma^* \neq \mathrm{id}$,(b) that $P'(\alpha) = \mathrm{id}$ whenever $\mathrm{AND}(\alpha) = 0$, and(c) that $P'(\alpha) = \sigma^*$ whenever $\mathrm{AND}(\alpha) = 1$. (c) follows from the fact that the only string α satisfying $\mathrm{AND}(\alpha) = 1$ is the all-ones string. Now, let $\alpha \in \{0,1\}^n$. Note that $P(\alpha) = \Pi_1(\alpha)\Pi_2(\alpha)$. If $\mathrm{AND}(\alpha) = 0$, then $P(\alpha) = \mathrm{id}$, so $\Pi_1(\alpha) = (\Pi_2(\alpha))^{-1}$, and so $P'(\alpha) = \mathrm{id}$ as well: this proves(b). Finally, note that $P(1, \dots, 1) \neq \mathrm{id}$, so $\Pi_1(1, \dots, 1) \neq (\Pi_2(1, \dots, 1))^{-1}$, and so $\sigma^* = P'(1, \dots, 1) \neq \mathrm{id}$, proving(a). □

Theorem3 (original comparison statement). For every function f, there is a read-4 permutation branching program of width $2^{n+2^{n-1}}$ and length $4n-4$ computing f.

4.2 Upper bounds

For our main upper bound, we modify our algorithm recreating the result of [11] (and analogously [12]) to have length $4n-4$. In particular, our program will read all but two variables four times, while the last two variables will be read twice.

Proof of Theorem3. First, we make an easy change to Theorem9 which allows us to achieve $4n-3$. Observe that in ToggleInput the order in which we add the inputs is irrelevant, and so consider ToggleMonomials where we reverse the order of toggling on Line6. Then notice that the last query on Line2 and the first query on Line6 are both made to x_n, and so we can merge these two layers along with our entire for loop (which reads no variables) into a single layer querying x_n. Moving to Algorithm1, this means we only query x_n twice, and furthermore the last query on Line1 and the first query on Line3 are both made to x_1, and so again by merging these two queries along with Line2 we only query x_1 three times. Now we will change our program so that x_1 is only read twice. Consider two new functions obtained by fixing the value of x_1, namely $f^0 = f(0, x_2 \dots x_n)$ and $f^1 = f(1, x_2 \dots x_n)$. Recall that we used the following polynomial to compute f, where $y_i = \tau_i^{\mathrm{in}} + x_i$:

$$q_f = \sum_{S,S' \subseteq [n]} c_{S,S'} \left(\prod_{i \in S} \tau_i^{\mathrm{in}} \right) \left(\prod_{i \in S'} y_i \right).$$

If we choose $b \in \{0,1\}$ and fix $\tau_1^{\text{in}} = 0$ and $y_1 = x_1 = b$, we get the following, which can be used to compute (since it is equal to) f^b:

$$q_{f^b} = \sum_{S,S' \subseteq [2..n]} (c_{S,S'} + b \cdot c_{S,S' \cup \{1\}}) \left(\prod_{i \in S} \tau_i^{\text{in}} \right) \left(\prod_{i \in S'} y_i \right).$$

We will use Algorithm1 to compute q_{f^b}, where $b = x_1$, by removing all reference to x_1 from ToggleInput and ToggleMonomials, and querying x_1 whenever we execute Lines2 or4 to determine whether to compute q_{f^0} or q_{f^1} in place of q_f. More specifically, ToggleInput will now only loop over $i = 2 \dots n$, while ToggleMonomials will now only loop over $S \subseteq [2..n]$. Finally in Algorithm1 we change Lines2 and4 to

$$R_{\text{out}} \leftarrow R_{\text{out}} \pm \sum_{S,S' \subseteq [2..n]} (c_{S,S'} + x_1 \cdot c_{S,S' \cup \{1\}}) \cdot \left(\prod_{i \in S} R_i^{\text{in}} \right) R_{S'}$$

where Line2 uses+ and Line4 uses−. Note that to execute these lines correctly, we will query x_1 and perform the corresponding instruction; thus we no longer ignore these two lines in calculating our program length. By our earlier definition of q_{f^b}, this exactly computes q_{f^b} for $x_1 = b$ as claimed. As above we will reverse the order of the queries in ToggleInput the second time it is called in ToggleMonomials, which allows us to read x_n only once per execution for a total of two reads. x_1 will be queried in Lines2 and4, and all other variables will be queried four times. Since we no longer use register R_1, or R_S when $1 \in S$, the number of registers is reduced from the original $n + 2^n$ in Theorem9 to $n - 1 + 2^{n-1}$. The output register R_{out} is now catalytic, so we add one register for a total $m = 2^{n+2^{n-1}}$. □

References

[1] Eric Allender, Anna Gál, and Ian Mertz. Dual VP classes. Comput. Complex., 26(3):583–625, 2017.

[2] David A. Mix Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in nc1. J. Comput. Syst. Sci., 38(1):150–164, 1989.

[3] Michael Ben-Or and Richard Cleve. Computing algebraic formulas using a constant number of registers. SIAM J. Comput., 21(1):54–58, 1992.

[4] Harry Buhrman, Richard Cleve, Michal Koucký, Bruno Loff, and Florian Speelman. Computing with a full memory: catalytic space. In Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014, pages 857–866. ACM, 2014.

[5] James Cook and Ian Mertz. Catalytic approaches to the tree evaluation problem. In Proccedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, Chicago, IL, USA, June 22-26, 2020, pages 752–760. ACM, 2020.

[6] James Cook and Ian Mertz. Encodings and the tree evaluation problem. Electron. Colloquium Comput. Complex., page 54, 2021. URL: https://eccc.weizmann.ac.il/report/2021/054.

[7] Stephen A. Cook, Pierre McKenzie, Dustin Wehr, Mark Braverman, and Rahul Santhanam. Pebbles and branching programs for tree evaluation. ACM Trans. Comput. Theory, 3(2):4:1– 4:43, 2012.

[8] Vincent Girard, Michal Koucky, and Pierre McKenzie. Nonuniform catalytic space and the direct sum for space. Electronic Colloquium on Computational Complexity (ECCC), 138, 2015.

[9] Frank Gray. Pulse code communication. https://patents.google.com/patent/US2632058A/ en, 1953. US Patent 2632058A.

[10] E.I. Nečiporuk. A boolean function. Dokl. Akad. Nauk SSSR, 169(4), 1966.

[11] Aaron Potechin. A note on amortized branching program complexity, 2017.

[12] Robert Robere and Jeroen Zuiddam. Amortized circuit complexity, formal complexity measures, and catalytic algorithms. In FOCS, pages 759–769. IEEE, 2021.